

WINDOW FUNCTIONS

A **window function** in SQL is a type of function that performs a calculation across a set of table rows that are somehow related to the current row. This is different from aggregate functions, which return a single result for a group of rows. Window functions allow you to perform aggregations **without collapsing the rows**.



WINDOW FUNCTIONS

Perform calculations (e.g. aggregation) on a specific subset of data, without losing the level of details of rows.

If we use **Group By** so in this, it is **summarising the multiple rows into a single** but in **window function** it is **not losing the level of details of rows**.

GROUP BY

ID	Product	Sales
1	Caps	10
2	Caps	30
3	Gloves	5
4	Gloves	20

Product	Total Sales
Caps	40
Gloves	25

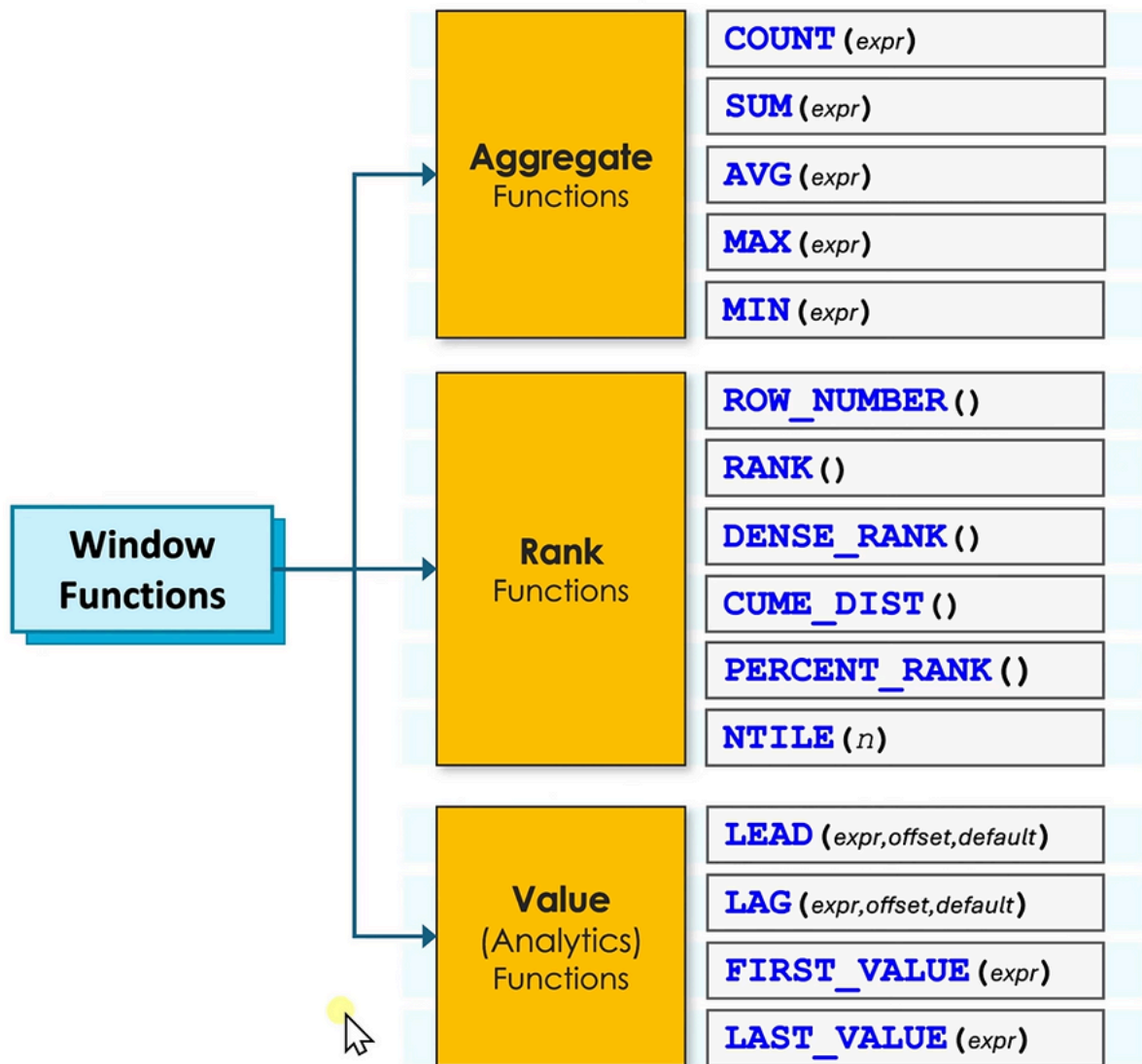
It **collapse** groups of rows into a single row
(Group-Level-Calculations)

Window Functions

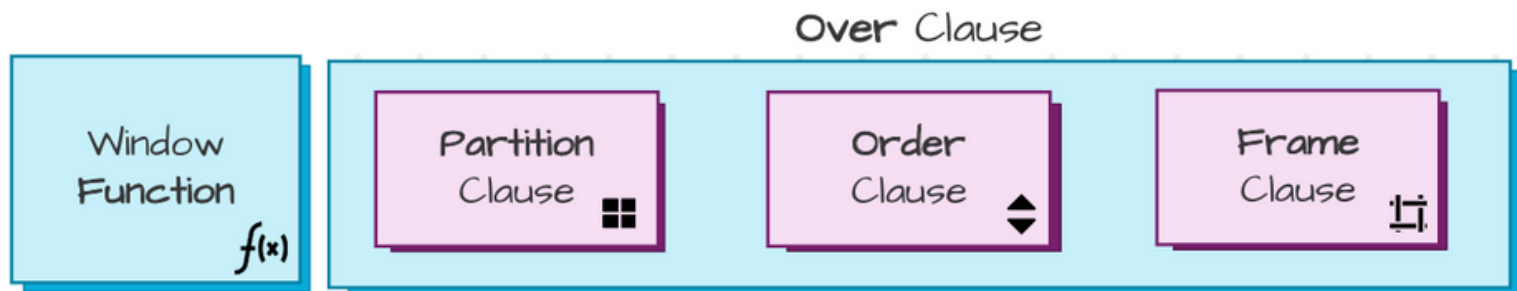
ID	Product	Sales
1	Caps	10
2	Caps	30
3	Gloves	5
4	Gloves	20

ID	Product	Total Sales
1	Caps	40
2	Caps	40
3	Gloves	25
4	Gloves	25

It **doesn't collapse** rows into a single row
(Row-Level-Calculations)



Window Syntax



```
AVG (Sales) OVER ( PARTITION BY Category ORDER BY OrderDate ROWS UNBOUNDED PRECEDING )
```

Calculation used
on the Window

f(x) Window
Function

Define the
Window

Over
Clause

AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate ROWS UNBOUNDED PRECEDING)

Function
Expression

Partition
Clause

Divides the dataset into
windows (Partitions)

Order
Clause

Sort the data
in a window

Frame
Clause

Define a subset of
rows in a window

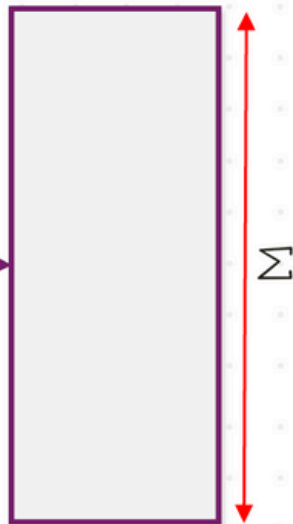
Partition By



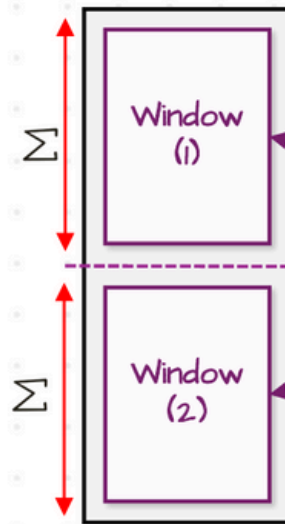
PARTITION BY divides the rows into groups, based on the column/s

SUM(Sales) OVER()

Calculation is done
on **entier Dataset**



SUM(Sales) OVER(PARTITION BY Product)



Calculation is
done **individually**
on each window

PARTITION BY divides the rows into groups, based on the column/s

Without
Partition By

Total sales across all rows (Entire Result Set)

```
SUM(Sales) OVER ()
```

Partition By
Single Column

Total sales for each Product

```
SUM(Sales) OVER (PARTITION BY Product)
```

Partition By
Combined-Columns

Total sales for each combination of Product and Order Status

```
SUM(Sales) OVER (PARTITION BY Product, OrderStatus)
```

16 • SELECT

```
17     currency,  
18     movie_id,  
19     unit,  
20     SUM(revenue) OVER() TotalSales,  
21     SUM(revenue) OVER(PARTITION BY unit, currency) TotalSalesByUnit_Currency  
22 FROM financials;
```

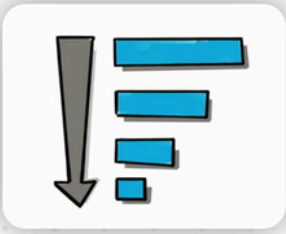
Result Grid

Filter Rows:

Export:

Wrap Cell Content:

currency	movie_id	unit	TotalSales	TotalSalesByUnit_Currency
INR	127	Thousands	153886.10	100000.00
INR	107	Millions	153886.10	34059.00
INR	108	Millions	153886.10	34059.00
INR	109	Millions	153886.10	34059.00
INR	128	Millions	153886.10	34059.00
INR	129	Millions	153886.10	34059.00
INR	130	Millions	153886.10	34059.00



ORDER BY

Sort the data within a window

(Ascending | Descending)

16 • **SELECT**

17 currency,

18 movie_id,

19 revenue,

20 **RANK()** **OVER**(**ORDER BY** revenue **DESC**) Rank_Revenue

21 **FROM** financials;

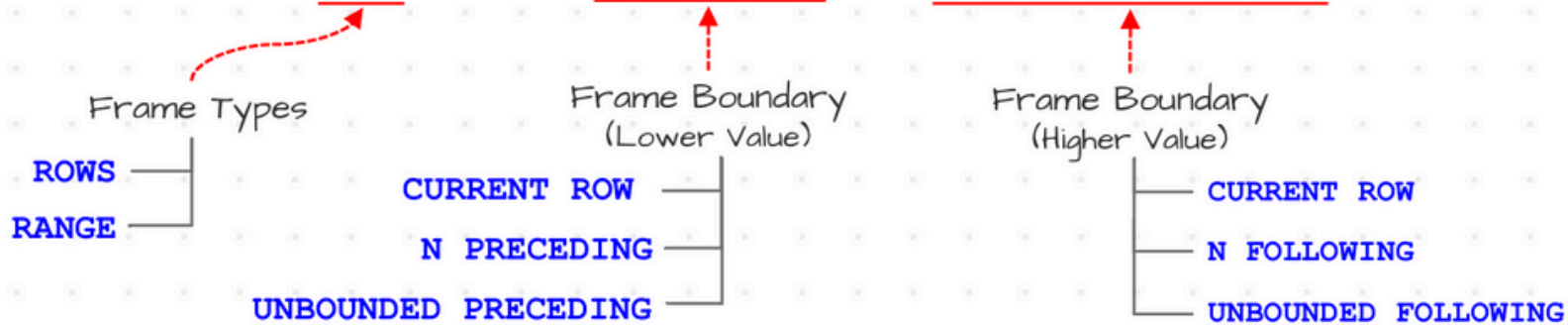
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

currency	movie_id	revenue	Rank_Revenue
INR	127	100000.00	1
INR	136	11690.00	2
INR	130	8540.00	3
INR	108	4000.00	4
INR	135	3409.00	5
USD	119	2847.00	6
USD	125	2798.00	7

Frame

AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)



Rules

- Frame Clause can only be used **together** with order by clause.
- Lower Value must be **BEFORE** the higher Value.

Window Rules



#1 RULE

Window functions can be used ONLY
in SELECT and ORDER BY Clauses

#2 RULE

Nesting Window Functions is not allowed!

#3 RULE

SQL execute WINDOW Functions after WHERE Clause

#4 RULE

Window Function can be used together with GROUP BY
in the same query, ONLY if the same columns are used

$f(x)$ Window Functions

Aggregate

- SUM ()
- AVG ()
- COUNT ()
- MAX ()
- MIN ()

Perform calculations on a set of rows and return a **single aggregated** value for each row

Rank

- ROW_NUMBER ()
- RANK ()
- DENSE_RANK ()
- NTILE ()
- CUME_DIST ()
- PERCENT_RANK ()

Assign a **rank** to each row in a window

Value

- LAG ()
- LEAD ()
- FIRST_VALUE ()
- LAST_VALUE ()

Return a **specific value** in a window to be compared with the value of **current row**

Aggregate Functions

AVG (Sales) OVER (PARTITION BY ProductID ORDER BY Sales)

Expression
is **required**
(Only **Numeric** Values)

Partition By
Is **Optional**

Order By
Is **Optional**

Aggregate Functions

		Expression	Partition Clause	Order Clause	Frame Clause
Aggregate Functions	COUNT (<i>expr</i>)	All Data Type	Optional	Optional	Optional
	SUM (<i>expr</i>)	Numeric Values			
	AVG (<i>expr</i>)	Numeric Values			
	MIN (<i>expr</i>)	Numeric Values			
	MAX (<i>expr</i>)	Numeric Values			

COUNT FUNCTION

Count the number of Rows
including NULLs

COUNT (*) **OVER** (PARTITION BY Product)

COUNT (1) **OVER** (PARTITION BY Product)

Product	Sales	Count
Caps	20	3
Caps	10	3
Caps	5	3
Gloves	30	3
Gloves	70	3
Gloves	NULL	3

This Row
is counted

Count the number of Rows
excluding NULLs

COUNT (Sales) **OVER** (PARTITION BY Product)

Column

Product	Sales	Count
Caps	20	3
Caps	10	3
Caps	5	3
Gloves	30	2
Gloves	70	2
Gloves	NULL	2

This Row
won't be
counted

3 • SELECT

```
4      brand_name,  
5      price,  
6  COUNT(*) OVER() Total_Brand,  
7  COUNT(*) OVER(PARTITION BY brand_name) Orderby_brand  
8  FROM smartphones;
```

Result Grid



 Filter Rows:

Export: 

Wrap Cell Content: 

brand_name	price	Total_Brand	Orderby_brand
apple	91999	973	46
apple	169900	973	46
apple	82990	973	46
apple	49990	973	46
apple	149900	973	46
apple	147900	973	46
apple	65999	973	46

SUM FUNCTION

SUM Function

Returns the **sum** of values in a window

SUM(Sales) **OVER**(PARTITION BY Product)

* Is **not** allowed!

Product	Sales	SUM	
Caps	20	35	20 + 10 + 5 = 35
Caps	10	35	
Caps	5	35	
Gloves	30	140	30 + 70 + 40 = 140
Gloves	70	140	
Gloves	40	140	

18 • SELECT

```
19     model,  
20     brand_name,  
21     SUM(price) OVER () Total_Price,  
22     SUM(price) OVER (PARTITION BY brand_name) Priceby_brand  
23 FROM smartphones;
```

model	brand_name	Total_Price	Priceby_brand
Apple iPhone 14 Plus (5...	apple	29945126	4414460
Apple iPhone 14 (512GB)	apple	29945126	4414460
Apple iPhone 12	apple	29945126	4414460
Apple iPhone 13 Pro Max...	apple	29945126	4414460
Asus ROG Phone 7	asus	29945126	522966
Asus ROG Phone 6 Batm...	asus	29945126	522966
Asus ROG Phone 5s 5G	asus	29945126	522966

Percentage of one phone prices to total prices of all the phones.

25 • SELECT

26 model,

27 brand_name,

28 SUM(price) OVER() totalprice,

29 price / SUM(price) OVER() * 100 Percentage

30 FROM smartphones;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

model	brand_name	totalprice	Percentage
OnePlus 11 5G	oneplus	29945126	0.1837
OnePlus Nord CE 2 Lite 5G	oneplus	29945126	0.0668
Samsung Galaxy A14 5G	samsung	29945126	0.0551
Motorola Moto G62 5G	motorola	29945126	0.0501
Realme 10 Pro Plus	realme	29945126	0.0835
Samsung Galaxy F23 5G ...	samsung	29945126	0.0568
Apple iPhone 14	apple	29945126	0.2204

AVERAGE FUNCTION

Default Average Function
exclude NULLs



AVG(Sales) OVER(PARTITION BY Product)

Product	Sales	AVG
Caps	20	
Caps	10	
Caps	5	
Gloves	30	50
Gloves	70	50
Gloves	NULL	50

For Caps: $\frac{20 + 10 + 5}{3} = ||$

For Gloves: $\frac{30 + 70}{2} = 50$

Deal with Nulls using COALESCE to
include NULLs



AVG(COALESCE(Sales,0)) OVER(PARTITION BY Product)

Replace NULL with 0

Product	Sales	AVG
Caps	20	
Caps	10	
Caps	5	
Gloves	30	50
Gloves	70	50
Gloves	0	50

For Caps: $\frac{20 + 10 + 5}{3} = ||$

For Gloves: $\frac{30 + 70 + 0}{3} = 33$

32 • **SELECT**

33 model,

34 brand_name,

35 **AVG**(price) **OVER** () TotalAVG_Price,

36 **AVG**(price) **OVER** (**PARTITION BY** brand_name) AVG_by_brand

37 **FROM** smartphones;

Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	model	brand_name	TotalAVG_Price	AVG_by_brand
▶	Apple iPhone 13 (512GB)	apple	30776.0802	95966.5217
	Apple iPhone 14 Pro Max...	apple	30776.0802	95966.5217
	Apple iPhone 15	apple	30776.0802	95966.5217
	Apple iPhone SE 4	apple	30776.0802	95966.5217
	Apple iPhone 15 Ultra	apple	30776.0802	95966.5217
	Apple iPhone 13 Pro (1TB)	apple	30776.0802	95966.5217
	Apple iPhone 14	apple	30776.0802	95966.5217

– Find a phone's price which greater than Average phone's price.

```

39 • SELECT
40      *
41  FROM(
42      SELECT
43          model,
44          brand_name,
45          price,
46          AVG(price) OVER() AvgPrice
47      FROM smartphones
48  ) AS sub
49      WHERE price > AvgPrice;

```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

model	brand_name	price	AvgPrice
OnePlus 11 5G	oneplus	54999	30776.0802
Apple iPhone 14	apple	65999	30776.0802

MIN/MAX FUNCTION

MIN

Returns the **minimum** value in a window

MIN(Sales) **OVER**(**PARTITION BY** Product)

Product	Sales	MIN
Caps	20	5
Caps	10	5
Caps	5	5
Gloves	30	30
Gloves	70	30
Gloves	40	30

5 is the lowest sales for Caps

30 is the lowest sales for Gloves

MAX

Returns the **maximum** value in a window

MAX(Sales) **OVER**(**PARTITION BY** Product)

Product	Sales	MAX
Caps	20	20
Caps	10	20
Caps	5	20
Gloves	30	70
Gloves	70	70
Gloves	40	70

20 is the highest sales for Caps

70 is the highest sales for Gloves

51 • **SELECT**

52 model,

53 brand_name,

54 MAX(price) **OVER** () MAX_Price,

55 MIN(price) **OVER** (**PARTITION BY** brand_name) MIN_by_brand

56 **FROM** smartphones;

model	brand_name	MAX_Price	MIN_by_brand
Apple iPhone 13 (512GB)	apple	199990	29990
Apple iPhone 14 Pro Max...	apple	199990	29990
Apple iPhone 15	apple	199990	29990
Apple iPhone SE 4	apple	199990	29990
Apple iPhone 15 Ultra	apple	199990	29990
Apple iPhone 13 Pro (1TB)	apple	199990	29990
Apple iPhone 14	apple	199990	29990

WINDOW RANKING FUNCTIONS



Month	Sales	Rank
Jun	70	1
Jul	40	2
Mar	30	3
Jan	20	4
Feb	10	5
Apr	5	6

Assign a Rank (Number)
for each Row



Top/Bottom N Analysis

ROW_NUMBER()
RANK()
DENSE_RANK()
NTILE()

Find Top 3 Products

Distribution Analysis

CUME_DIST()
PERCENT_RANK()

Find Top 20% Products



Products	Sales
E	70
B	30
A	20
C	10
D	5

Integer-based Ranking

Discrete Values

1
2
3
4
5

Percentage-based Ranking

Continuous Values

0
0,25
0,5
0,75
1

Ranking Function

RANK () OVER (PARTITION BY ProductID ORDER BY Sales)

Expression
must be **empty**

Partition By
Is **Optional**

Order By
Is **required**

		Expression	Partition Clause	Order Clause	Frame Clause
Rank Functions	ROW_NUMBER ()	Empty	Optional	Required	Not allowed
	RANK ()				
	DENSE_RANK ()				
	CUME_DIST ()				
	PERCENT_RANK ()				
	NTILE (n)	Number			

Rank Functions	ROW_NUMBER ()	Assign a unique number to each in a window	ROW_NUMBER () OVER (ORDER BY Sales)
	RANK ()	Assign a rank to each row in a window, with gaps	RANK () OVER (ORDER BY Sales)
	DENSE_RANK ()	Assign a rank to each row in a window, without gaps	DENSE_RANK () OVER (ORDER BY Sales)
	CUME_DIST ()	calculates the cumulative distribution of a value within a set of values	CUME_DIST () OVER (ORDER BY Sales)
	PERCENT_RANK ()	Returns the percentile ranking number of a row.	PERCENT_RANK () OVER (ORDER BY Sales)
	NTILE (n)	Divides the rows into a specified number of approximately equal groups	NTILE (2) OVER (ORDER BY Sales)

ROW_NUMBER

- It doesn't handle ties.

Assign a **unique sequential integer** to each row with in a window

```
ROW_NUMBER() OVER (ORDER BY Sales DESC)
```

Sales	Rank
100	1
80	2
80	3
50	4
20	5

We have a **tie** here!

Row_Number() assigns a **unique rank** to each of row

RANK

Assign a **rank** to each row with in a window

```
RANK() OVER (ORDER BY Sales DESC)
```

Sales	Rank
100	1
80	2
80	2
50	4
20	5

We have a **tie** here!

RANK () assigns a **same rank** for both of them

RANK () leaves a **GAP in Ranking** after a tie

58 • SELECT

59 model,

60 brand_name,

61 price,

62 RANK() OVER (ORDER BY price DESC) Price_Rank

63 FROM smartphones;

model	brand_name	price	Price_Rank
Apple iPhone 15 Pro Max	apple	142990	13
Apple iPhone 14 Pro Max...	apple	139990	14
Apple iPhone 12 Pro (51...	apple	139900	15
Apple iPhone 13 Pro Max...	apple	139900	15
Sony Xperia Pro-I	sony	134999	17
OPPO X 2021	oppo	134999	17
Apple iPhone 15 Pro	apple	130990	19

DENSE_RANK

Assign a rank to each row with in a window, but does not leave gaps in the ranking

DENSE_RANK() OVER (ORDER BY Sales DESC)

Sales	Rank
100	1
80	2
80	2
50	3
20	5

We have a tie here!

DENSE_RANK() assigns a same rank for both of them

DENSE_RANK() doesn't leaves a GAP in Ranking after a tie

58 • SELECT

```

59     model,
60     brand_name,
61     price,
62     DENSE_RANK() OVER (ORDER BY price DESC) Price_Rank
63 FROM smartphones;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

model	brand_name	price	Price_Rank
Apple iPhone 15 Pro Max	apple	142990	13
Apple iPhone 14 Pro Max...	apple	139990	14
Apple iPhone 12 Pro (51...	apple	139900	15
Apple iPhone 13 Pro Max...	apple	139900	15
Sony Xperia Pro-I	sony	134999	16
OPPO X 2021	oppo	134999	16
Apple iPhone 15 Pro	apple	130990	17

ROW_NUMBER()

ROW_NUMBER() OVER(ORDER BY Sales DESC)

Sales	Rank
100	1
80	2
80	3
50	4
20	5

Unique Rank

Does NOT handle Ties

No Gaps in Ranks

RANK()

RANK() OVER(ORDER BY Sales DESC)

Sales	Rank
100	1
80	2
80	2
50	4
20	5

Shared Rank

Handles Ties

Gaps in Ranks

DENSE_RANK()

DENSE_RANK() OVER(ORDER BY Sales DESC)

Sales	Rank
100	1
80	2
80	2
50	3
20	4

Shared Rank

Handles Ties

No Gaps in Ranks

Using ROW_NUMBER we can find the top model by highest price in each model but using RANK or DENSE_RANK it is not possible sometimes because it gives shared Rank.

```

66 • SELECT *
67 FROM (
68 SELECT
69     model,
70     brand_name,
71     price,
72     ROW_NUMBER() OVER (PARTITION BY brand_name ORDER BY price DESC) Rankbyprice
73 FROM smartphones
74 ) AS top_models
75 WHERE Rankbyprice = 1;

```

model	brand_name	price	Rankbyprice
Apple iPhone 14 Pro Max...	apple	182999	1
Asus ROG Phone 6D Ulti...	asus	107990	1
Blackview BV5200 Pro	blackview	8990	1
BLU F91 5G	blu	14990	1

NTILE ()

- Divides the rows into a specified number of approximately equal groups (Buckets).
- Larger Groups comes first that's why 3 rows in 1 bucket and then 2 one.

Divides the rows into a specified number of approximately **equal groups** (buckets)

NTILE(2) OVER (ORDER BY Sales DESC)

Number of Buckets

	Sales	NTILE (2)
Bucket (1)	100	1
	80	1
Bucket (2)	80	1
	50	2

$$\text{Bucket Size} = \frac{\text{Number of Rows}}{\text{Number of Buckets}}$$

(Nr of Rows in each Bucket)

$$\text{Bucket Size} = \frac{4}{2} = 2$$

Divides the rows into a specified number of approximately **equal groups** (buckets)

```
NTILE(2) OVER (ORDER BY Sales DESC)
```

Number of Buckets

Sales	NTILE (2)
100	1
80	1
80	1
50	2
20	2

Bucket (1)

Bucket (2)

$$\text{Bucket Size} = \frac{\text{Number of Rows}}{\text{Number of Buckets}}$$

(Nr of Rows in each Bucket)

$$\text{Bucket Size} = \frac{5}{2} = 2.5$$

Larger groups come **first** then smaller groups

78 • SELECT

79 model,

80 brand_name,

81 price,

82 NTILE(10) OVER (ORDER BY price DESC) OneBucket

83 FROM smartphones;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

model	brand_name	price	OneBucket
Xiaomi Mi Mix Alpha	xiaomi	199990	1
Apple iPhone 14 Pro Max...	apple	182999	1
Apple iPhone 13 Pro Max...	apple	179900	1
Apple iPhone 14 Pro (1TB)	apple	172999	1

1. CUME_DIST() — Cumulative Distribution

- Calculates the relative position of a row including itself and all rows before it.
- Formula:

$$\text{CUME_DIST} = \frac{\text{Number of rows with value} \leq \text{current row}}{\text{Total number of rows in partition}}$$

- Always gives values between $0 < x \leq 1$.

2. PERCENT_RANK() — Relative Rank

- Calculates the relative rank of a row among all rows in the partition.
- Formula:

$$\text{PERCENT_RANK} = \frac{\text{Rank} - 1}{\text{Total rows} - 1}$$

- Minimum value is 0 (for the lowest rank).
- Maximum is 1, but it does not include the current row like CUME_DIST does.

Calculates the relative position of a specified value in a group of values.

`CUME_DIST() OVER (ORDER BY Sales)`

Sales	Dist
20	0,2
50	0,4
60	0,6
80	
100	

Current Row



$$\text{CUME_DIST}(x) = \frac{\text{Number of Rows less than or equal to } x}{\text{Total Number of Rows}}$$

$$\text{CUME_DIST}(60) = \frac{3}{5} = 0,6$$

Returns the **percentile ranking** number of a row

PERCENT_RANK() OVER (ORDER BY Sales)

Sales	Rank	Dist
20	1	0
50	2	0,25
60	3	0,5
80	4	
100	5	

Current Row



$$\text{PERCENT_RANK}(x) = \frac{\text{Rank of } x - 1}{\text{Total Number of Rows} - 1}$$

$$\text{PERCENT_RANK}(60) = \frac{2}{4} = 0.5$$

WINDOW VALUE FUNCTIONS

Value
(Analytics)
Functions

LEAD (expr, offset, default)

Returns the value from a previews row

LEAD (Sales, 2, 0) OVER (ORDER BY OrderDate)

LAG (expr, offset, default)

Returns the value from a subsequent row

LAG (Sales, 2, 0) OVER (ORDER BY OrderDate)

FIRST_VALUE (expr)

Returns the first value in a window

FIRST_VALUE (Sales) OVER (ORDER BY OrderDate)

LAST_VALUE (expr)

Returns the last value in a window

FIRST_VALUE (Sales) OVER (ORDER BY OrderDate)

Expression

Partition
Clause

Order
Clause

Frame
Clause

All Data Type

Optional

Required

Not allowed

Optional

Should be
used

Value
(Analytics)
Functions

LEAD (expr, offset, default)

LAG (expr, offset, default)

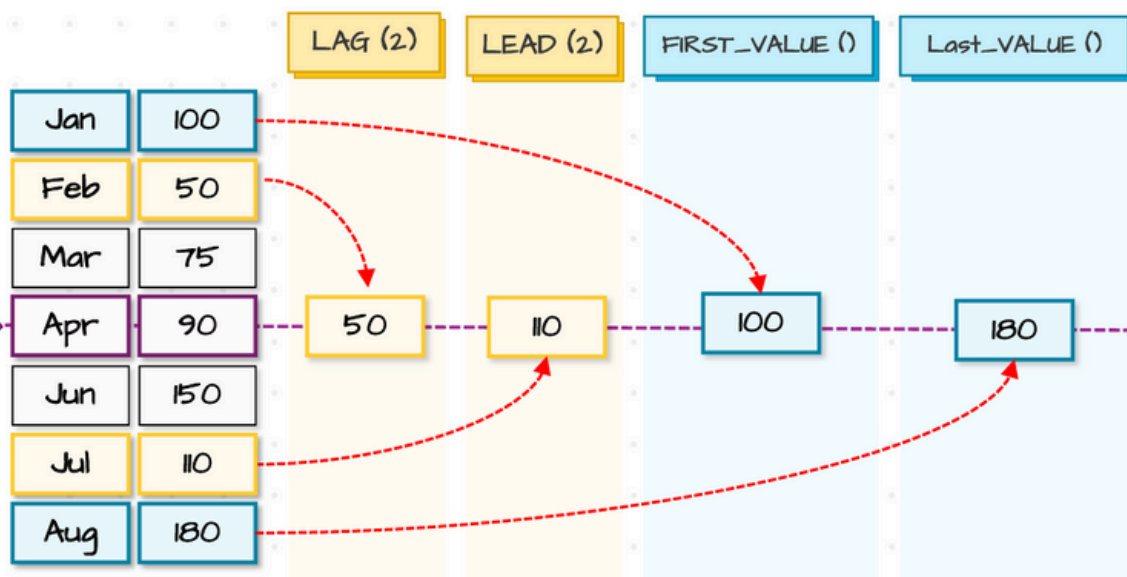
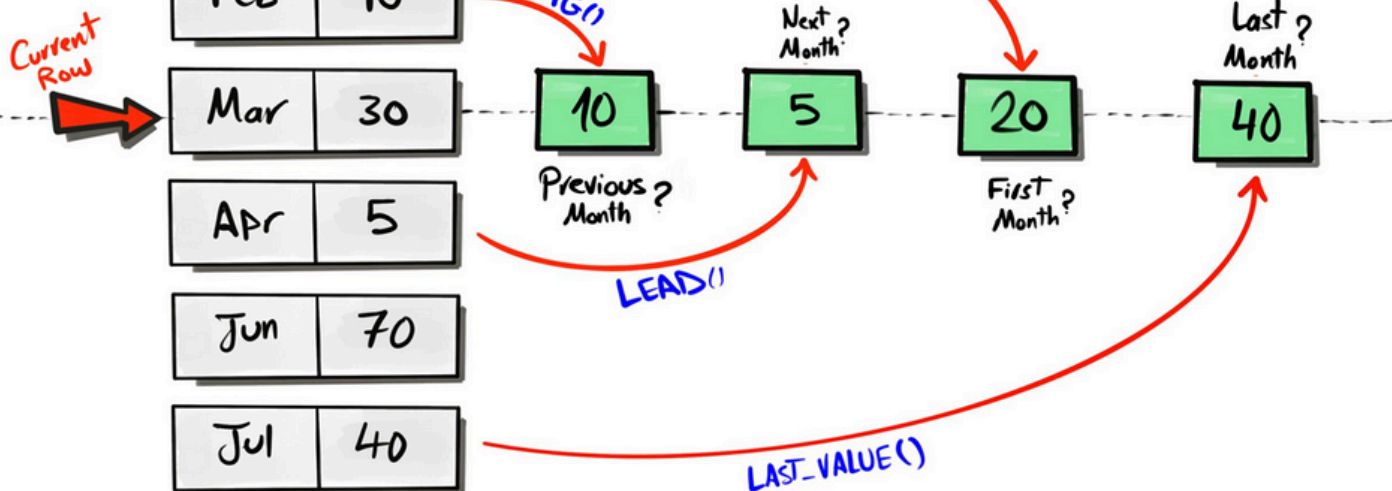
FIRST_VALUE (expr)

LAST_VALUE (expr)

HOW IT WORKS?

Month	Sales
Jan	20
Feb	10
Mar	30
Apr	5
Jun	70
Jul	40

Access a value from other Row



Partition By
Is **Optional**

Order By
Is **Required**

LEAD(Sales, 2, 10) OVER(PARTITION BY ProductID ORDER BY OrderDate)

Expression
is **required**
(Any Data Type)

Default Value (Optional)
Returns default value if next/previous row is not available!
Default = **NULL**

Offset (Optional)
Number of rows forward or backward from current row
default = **1**